

Lecture 8

JavaScript Basics

- Variables:
 - var, let, const
 - data types
 - 7 primitive data types:
string, number, bigint, boolean, undefined, symbol, and null
 - built-in objects
- Arrays
 - declaration, assigning values, indexing
- Operators
- Control structures
 - if, switch, for, while, do,
- Functions
 - declaration, parameters
 - arrays as parameters
 - scope
 - return values
 - anonymous functions
 - callback function

JavaScript

- **dynamically** and **loosely typed** *scripting* language
- enables you to dynamically update content, control multimedia, animate images, etc.
- heavily exploits asynchronous callbacks
- works with a single thread
- prominent functions!

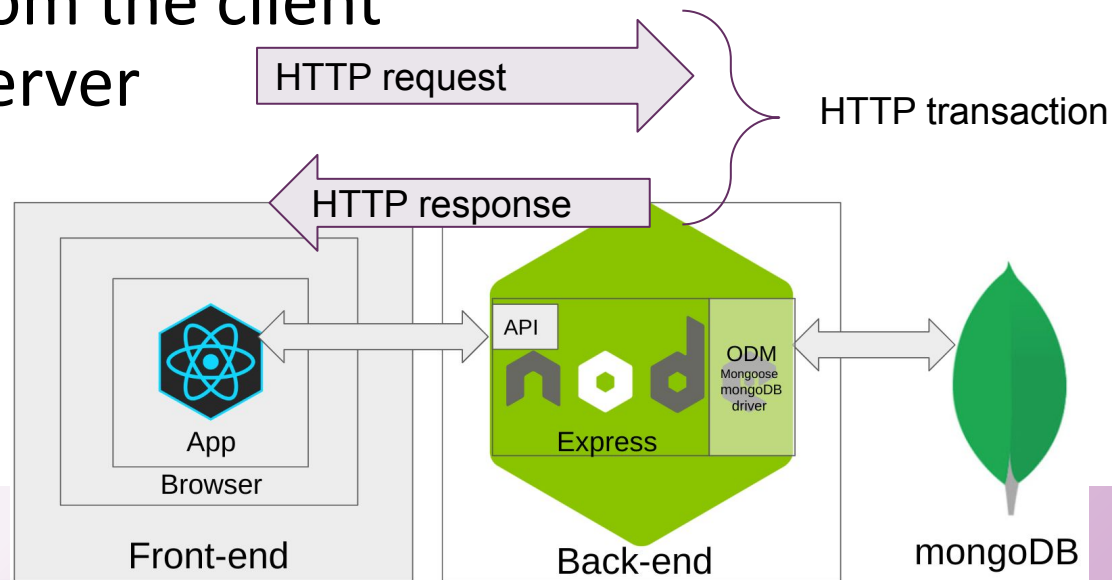
Role of JavaScript

○ Front-end JavaScript

- interpreted by the browser
- deals with user input/events
- changes the state of the page
- fetches data from other servers
- often send data from the client (browser) to the server

○ Back-end

- i.e., server side applications using Node.js
 - see, e.g. [Nicholas Zakas](#)
 - .. same topic in Finnish, [Tuomas Kumpula](#)



JavaScript, frameworks and libraries

- An implementation of ECMAScript standard
 - current version of ECMAScript is 9, EcmaScript 2018
 - an extensive number of new additions in v6
- Abundant ‘function’ collections and other program constructs
 - some called as ‘frameworks’, others as ‘libraries’
 - such as jQuery and d3, the latter is used in the assignment
 - exploitable in your own JavaScript program

Adding scripts on a page

HTML

```
...
<link rel="stylesheet" href="css/js.css" />

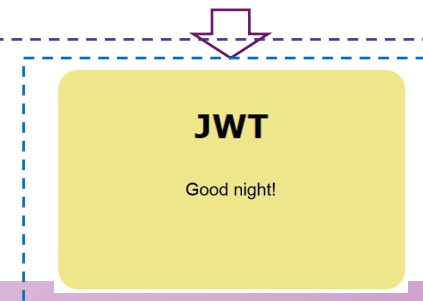
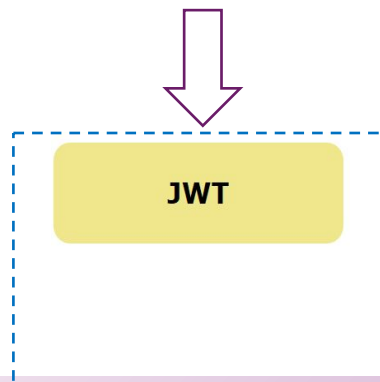
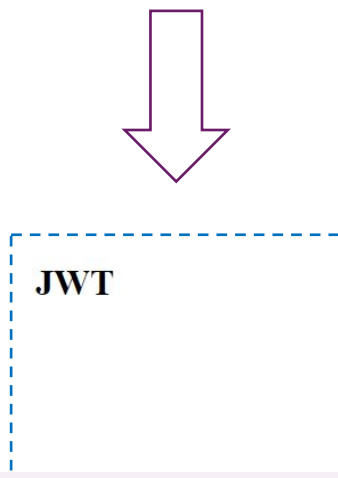
<body>
  <div id="container">
    <h1>JWT</h1>
    <p class="greet"></p>
  </div>
  <script src="script.js"> </script>
</body>
...
```

CSS (js.css)

```
#container {
  /* narrow centered,
  colored element */
  font-family: Verdana,
               sans-serif;
  background-color: khaki;
  margin: 1em auto;
  width: 20em;
  text-align: center;
  padding: 1em;
  border-radius: 20px;
}
```

JavaScript (script.js)

```
function greet() {
  const elem=document.querySelector("p.greet");
  const greetText = "";
  const inHours = new Date().getHours;
  if (inHours > 21 || inHours < 1) {
    greetText = 'Good night!';
  } else if (inHours > 18) {
    greetText = 'Good evening!';
  } else if (inHours > 11) {
    greetText = 'Good afternoon!';
  } else if (inHours > 5) {
    greetText = 'Good morning!';
  } else {
    greetText = 'You should be sleeping!';
  }
  greetElem.innerHTML = greetText;
}
/* ===== */
greet();
```



02 greeting

CDN (Content Delivery Network)



- Set of servers, which are optimized to return static files fast from nearby
- Your browser may have the library in its cache, so it doesn't ask for it
- You can get Bootstrap online using the following `<link>` element:

```
<link rel="stylesheet"
href="https://stackpath.bootstrapcdn.com/bootstrap/4.1.0/css/bootstrap.min.css"
integrity="sha384-9gVQ4dYFwwWSjIDZnLEWnxCjeSWFphJiwGPXr1jddIhOegiu1FwO5qRGvFXOdJZ4"
crossorigin="anonymous">
```

... and the JavaScript library online using the following `<script>` element:

```
<script src="https://stackpath.bootstrapcdn.com/bootstrap/4.1.0/js/bootstrap.min.js"
integrity="sha384-uefMccjFJAIv6A+rW+L4AHf99KvxDjWSulz9VI8SKNVmz4sk7buKt/6v9KI65qnm"
crossorigin="anonymous"></script>
```

Comments in JavaScript

```
// this is a comment
```

```
/* this is a multi-line  
comment */
```

○ Comments

- to explain what the code does
- to outcomment code for debugging

○ JSDoc

- for function descriptions
- VSCode auto-completes “/**” <Enter>
- Note! JSDoc is the required documentation of the coursework assignment

```
/**  
 * @param {string} student  
 * @returns {string} greeting for a student  
 * @description Return a greeting as a  
 * template string. For demo purposes only.  
 */  
function demoJSDocs (student) {  
    return `Hello ${student}, WebDev  
wanna-be`;  
}
```

Variables

- Definition and initialisation can be done apart from each other, i.e., **loosely**, or be combined

```
let count, name;           // variable does not have data type yet
count = 0;                 // number variable
name = "Charles Dickens";  // string variable
let number2 = 12;          // new number variable
name = 123;                // changed into an integer variable (!)
name = "New name";         // changed back to string, not allowed by const
```

- The data type of a variable is determined **dynamically** based on the assigned value
- Can a variable be used without defining it first?
 - yes, in this case the definition is made automatically
 - it becomes a global variable

Unless JavaScript
interpreted in strict
mode

Variables: var

- Older way of defining variables
 - declared in function,
=>function-scope
 - declared outside a function,
=> global scope

```
var a = 5;  
var sum = 0;  
  
function doSomething () {  
    var b = 12; // local variable  
    c = 10; // global (!)  
    ....  
}  
/* here b does not exist,  
   but c does */
```

Hoisting of var

- When `var` is used
 - the declaration of a variable is 'hoisted' to the top of the function
- Use `let` or `const` rather than `var`

```
function getValue(condition){  
    if(condition){  
        var color = "blue";  
        // other code  
        return color;  
    }  
    else{  
        // color exists here with  
        // value undefined  
    }  
}
```



new in ES6

a convention:
JavaScript identifiers in
camelCase

Variables: let

- we always declare a variable **the first time** it appears

- `let price;`
- `let quantity;`
- `let total;`

- a declared variable can be assigned

- `price = 16;`

- block scope: restricted within the block statement - `{ }`

```
function getValue(condition) {  
    if(condition) {  
        let color = "blue";  
        //other code  
        return color;  
    }  
    else {  
        // color does not exist here  
    }  
    // color does not exist here  
}
```

new in ES6

Variables: const

- declare once - re-assignment not permitted!

```
const PI = 3.141592653589793;
```

```
PI = 3.14;           // This will give an error
```

```
PI = PI + 10;        // This will also give an error
```

https://www.w3schools.com/js/js_const.asp

- block-scope, like let
- beneficial in targeting immutability
 - no unintended type changes

Identifier (name)

- JavaScript is case-sensitive
- Variable names can contain
 - letters, numbers, underscores (varName, this_one_too)
 - cannot start with a number: 99_notAllowed, isAllowed_99
- Reserved word can not be used as variable names, e.g.,
https://www.w3schools.com/js/js_reserved.asp

Data types

- Primitive data types

- `number`, `string`, `boolean`, `undefined` (+ rare: `bigint`, `symbol`), `null`

- Objects

- a set of ready defined built-in objects:

- `Number`, `String`, `Boolean`, `Array`, `Date`,
`Function`, `Math`, `RegExp`

- Constants

- `const ROWS = 12; /* constant names with caps, a convention*/`

Built-in objects

○ Utility objects

- ready-made objects with lots of useful methods for data manipulation
- e.g., Math, Date and RegExp

```
let seed = Math.random(); // random number between 0 and 1 [0,1)
let lottoNr = Math.floor(1 + 40*seed);
const start_date = new Date(2019,4,4,14,15); //year, month, day, hour, and minute
const month = start_date.getMonth(); // month = 4
let today = new Date();
const pattern = /[0-9]/;
const str = new String(today);
let found = str.match(pattern); // returns the first match
```

string

- written in "quotation marks", 'apostrophes' or `backticks` (= template literals)
- quotations and other special chars need to be escaped with backslash (\) escape character:
 - \" - quotation mark
 - \' - apostrophe
 - \\ - backslash
- other common escape characters:
 - \n - line break
 - \t - tabulator

```
const str1 = 'This is a string';  
const str2 = "So is this one";  
const str3 = 'Here is  "a quote", that is why both are used';  
const str4 = "Here is \"a quote\", it can be done like this, as well";  
const str5 = "123"; // this is a string, not a number  
const str6 = "first row\nsecond row"; // would print on two rows
```


Concatenation

- When the other of operands is a string
 - the plus (+) operator taken as a concatenation operator
 - it combines strings; also the other operand is converted into string
- template string (literals in ES6) enclosed by backticks, `\${str7}`, an option for easy variable embedding

```
const start = "This is the start";
const end = "and this the end";
const mfono = start + ", " + end + "."; // "This is the start and this the end"
const sosSecNr = "120289-123C";
const year = sosSecNr.substr(4, 2); // year on "89"
const bYear = "19" + year; // string "1989"
const year2 = year + 1; // string "891"
const year3 = parseInt(year) + 1; // number 90, parseInt() global function
const year4 = Number(year)+1; // number 90, Number() global function
let count = 12; // number 12
    count = 12 + ""; // string "12"
const Dude = "Zelensky❤️";
const str7 = `Hi ${Dude}, literals are strong weapons!` // renders Dude's value to a string
```

Boolean

○ Values: `true` ja `false`

```
const married = true;  
if (married) {  
    console.log("Let us be very married");  
}  
const b = (nr == 1000);  
const notBoolean = "false"; // a string variable
```

- `truthy` and `falsy` values are considered `true` and `false` in Boolean context
 - i.e., in `==` comparisons, where the second value is auto-converted to the same type as the first one, called type coercion.
 - no type coercion with `===` comparison (a so-called strict comparison)

undefined

- The data type of an variable is defined by its contents
- How about a variable, which does not have value (yet)?
`undefined`
 - undefined is a data type which contains only one value (value undefined)
- Value of a variable, if
 - no value given and it could be initiated as `null`
 - `null` in turn can be auto-casted into an empty string, `null` or `false` (depends on the receiving variable)

Array

- An array is represented by the **Array** object. To create an array of N elements, you can write

```
let myArray = new Array(N);
```

- Index of an array runs from 0 to N-1.
- JavaScript lets you store values of different types
- Property `length` tells the number of elements in the array.
- Consists of various methods to manipulate its elements. e.g., `reverse()`, `push()`, `concat()`, `flat()` etc.



Array

Array Declaration Examples

```
// An array of 3 elements, each element is undefined  
let Car = new Array(3);
```

```
// assign values to the elements  
Car[0] = "Ford";  
Car[1] = "Toyota";  
Car[2] = "Honda";
```

```
// Create an array of three elements with initial values  
let Car2 = new Array("Ford", "Toyota", "Honda");
```

```
// Create an array of three elements with initial values  
let Car3 = ["Ford", "Toyota", "Honda"];
```

Assigning values to Arrays

```
// An array of 3 elements with initial values of different  
types
```

```
let tmp3 = new Array(1, "a", true);
```

```
// Makes tmp3 an array of 10 elements
```

```
tmp3.length = 10; // from tmp[3] to tmp[9] undefined.
```

```
// Makes tmp3 an array of 100 elements
```

```
tmp3[99] = "Something";
```

```
// tmp[3] to tmp[98] are undefined.
```

Indexing array

- the index of an element is used to access each item in the array
- this is a value between 0 and the length - 1
- e.g, an array `weekDays` with 7 elements is labelled
 - `weekDays[0], weekdays[1], ... weekDays[6]`
 - `weekDays.length` has a value 7
- a value stored in a variable to access an array element
 - `let counter = 3;`
 - `let someday = weekDays[counter];`

typeof()

- this is a built-in function that returns the type of data currently stored in a variable, or in an expression

```
let n = 6, name = "TAU";
```

```
typeof(n)      returns 'number' (as a string)
```

```
typeof(name)   returns 'string' (as a string)
```

```
typeof(6 > 5)  returns 'boolean' (as a string)
```

```
let title;
```

```
typeof(title)  returns 'undefined' as a string
```

```
typeof(arrOfSomething) returns object (not an array)
```

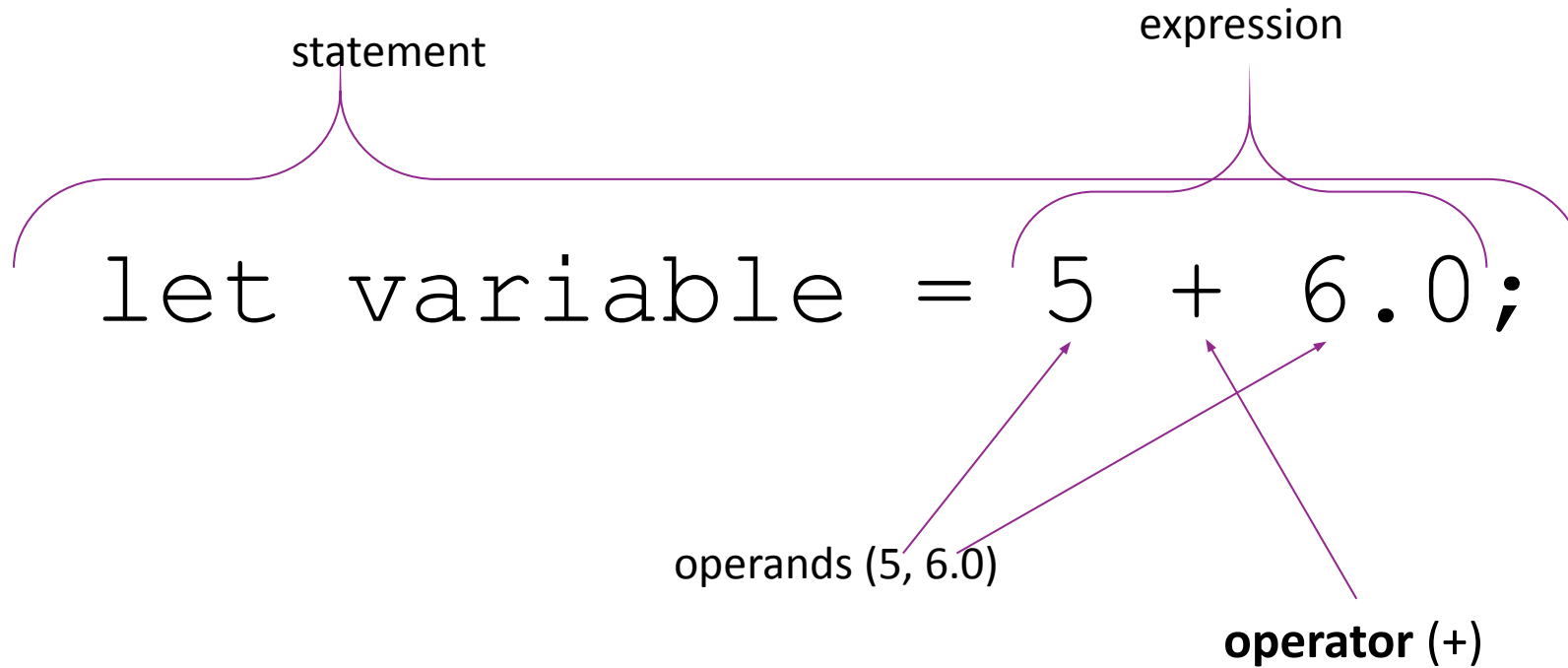

Checking if a parameter is an array

- How do we know that what has been passed to the function is an array?

```
function addArrayItems(arrOfSomething) {  
  if(Array.isArray(arrOfSomething)) {  
    // all good, it's an array  
  }  
}
```

`typeof(arrOfSomething)` returns object (not an array)

Expressions: operators



Arithmetic and comparison operators

- Arithmetic operators

`+, -, *, /, %, ++, --`

```
// let's assume variables  
// a and b are defined:  
let a = 5;  
let b = 10;
```

```
a + b    // value is 15  
a - b    // value is -5  
a * b    // value is 50  
a / b    // value is 0.5  
a % b    // value is 5  
a++      // value is 5  
a--      // value is 5
```

- Comparison operators :

`==, ===, !=, !==`

```
// let's assume the  
// definitions:  
let a = 5;  
let b = 10;  
let c = '10';
```

```
(a == b) // value is false  
(b == c) // value is true  
(b === c) // value is false  
(b != c) // value is false  
(b !== c) // value is true  
(a > b) // value is false  
(a < c) // value is true  
(b > c) // value is false  
(b < c) // value is false
```

- `<, >, <=, >=`

Logical and binary operators

- Logical operators (and, or, not): `&&`, `||`, `!`

```
// let's assume the definitions:
let a = true;
let b = false;

(a && b) // value is false
(a || b) // value is true
!(a && b) // value is true
```

- Binary operators:
`&`, `|`, `~`, `^`, `<<`, `>>`, `>>>` (bitwise AND, OR, NOT, XOR, left-, right-, unsigned right shift)
 - manipulate integer operands binary representations
 - for example, `|` is a binary or, which performs a logical or-operation for the corresponding bits of the operands binary representation

Assignment operators

- Assignment operators
=, +=, -=, *=, /=, %=

```
// assuming the declarations:  
let a = 5;  
let b = 10;
```

```
c = a + b; // c is 15
```

```
b += a;    // b is 15
```

```
b -= a;    // b is 5
```

```
b *= a;    // b is 50
```

```
b /= a;    // b is 2
```

```
b %= a;    // b's value is 0
```

Control structures: `if ( ) { . . }`

```
if (condition) {  
    statement;  
}  
[else {  
    statement;  
}]
```

```
let age;  
.  
.  
.  
    if (!age) {  
        ageGroup = "infant";  
        // ...  
    }  
    else if (age <30) {  
        ageGroup = "teen";  
        // ...  
    }  
    else if (age <65) {  
        ageGroup = "adult";  
        // ...  
    }  
    else ageGroup = "senior";
```

switch() { ... }

- break-statement makes the control leave the structure

```
switch (expression) {  
    case val1: statement;  
                [statement;]*  
                break;  
    case val2: statement;  
                [statement;]*  
                break;  
    ...  
    case valN: statement;  
                [statement;]*  
                break;  
    default: statement(s);  
}
```

```
switch (nr) {  
    case 10: alert("Number is 10");  
             break;  
    case 20: alert("Number is 20");  
             break;  
    default: alert("Nope, neither 10 nor 20");  
}
```

for () { .. }

- Statements in the loop performed "as long as" the condition holds

```
for (startStatement; loopCondition; change in the loop variable) {  
    statement;  
    [statement;]*  
}
```

```
for (let i=0; i<10; i++) {  
    sum = sum + table[i,j];  
}
```


while () { ... }

- Statements performed as long as the condition holds (is true)

```
while (condition) {  
    statement;  
    [statement;]*  
}
```

- break
 - execution point leaves the structure
 - brings the control out from any block
- continue
 - within a loop, transfers execution immediately to the condition
 - if condition is true, the execution continues from the beginning of the next loop

```
let i=10;  
while (i>0) {  
    console.log(i+" ");  
    i--;  
}
```

```
let i=10;  
while (i>0) {  
    if (n===0) {  
        break;  
    }  
    console.log(i+i/n);  
    i--;  
}
```

do { ... } while ();

```
let count = 0;
console.log("Starting Loop");
do
{
    document.write("Current Count : " + count);
    count++;
} while (count < 5);
console.log("Loop stopped!");
```

- ensures that **the code is executed at least once**
- do not miss a semicolon at end of do .. while loop

Functions in JavaScript

- simple function to square a number

```
function square(value) {  
    return value * value;  
}
```

- Can be called as a part of an expression

```
const radius = 6;  
const area1 = 3.1459 * square(radius);  
const someNumber = 3 + square(6 * radius);
```

Declarations vs. expressions

○ Function declaration

```
function square(value) {  
    return value * value;  
}
```

○ Function as an expression

```
let square = function(value) {  
    return value * value;  
};
```

- the function is *anonymous* as the function has no name

Anonymous functions (lambdas)

```
let z = function(value){  
    return value * value;  
} /* function expression */  
let side1 = 3;  
let newArea = z(side1);  
  
let x = z;  
let myArea = x(side1 + 6);
```

- we can assign the value of `z` to a new variable `x`
- `x` now contains a callable function

Declaration hoisted unlike expression



```
hoisted();  
hoistedExp();  
hoistedExp2();
```

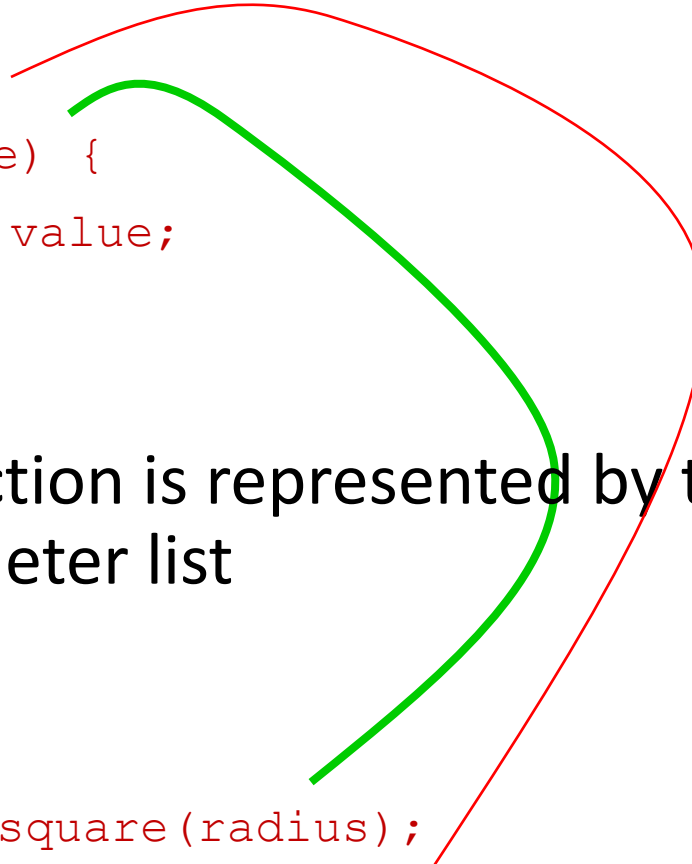
```
function hoisted() {  
  console.log('declaration is being hoisted');  
}
```

```
var hoistedExp = function() {  
  console.log('expression is not');  
}
```

```
let hoistedExp2 = function() {  
  console.log('expression is not');  
}
```

Passing data to a function

```
function square(value) {  
    return value * value;  
}
```



- data passed to a function is represented by the (local) variable in the parameter list

```
let radius = 6;  
let area1 = 3.1459 * square(radius);  
let someNumber = 3 + square(6 * radius);
```

Parameter lists

- the parameters passed to the function when it is called are known as *actual parameters (arguments)*
- the corresponding parameters inside the function are known as *formal parameters (or function parameters)*
- both are separated by *commas*

```
function product(val1, val2) {  
    return val1 * val2;  
}
```

2 formal
parameters

```
let width = 5, height = 6;  
let area = product(width, height);
```

2 actual
parameters

```
let bigArea = product(10*width, 10*height);
```

2 actual
parameters

Variable number of parameters

- possible to enter fewer or more actual parameters than formal parameters

```
function product(val1, val2) {  
    return val1 * val2;  
}  
const x = product (1);  
const y = product (3,4,5,6);
```

- missing parameters are *undefined*
- additional parameters are ignored

Variable numbers of *arguments*

- "arguments" is a local variable (array-like object) available in every function
- access the arguments either through the formal parameters or through the "arguments" object.

```
function add ()  
{  
  let s = 0;  
  for (let i = 0; i < arguments.length; i++)  
    s += arguments[i];  
  return s;  
}
```

'arguments' is a pre-defined object available in all functions

```
add(1, 2, 3);           // returns 6  
add(1, 2, 3, 4, 5);     // returns 15  
add(1, 2, "3", 4, 5);   // what does this return?
```

Default values of parameters

- We can provide values for the formal parameters in case the actual parameters are missing

```
function product(val1 = 10, val2 = 5) {  
    return val1 * val2;  
}  
let x = product (3,4);  
let y = product (3);  
let z = product (undefined, 6);
```

- If parameters are provided, default values are ignored
- Missing parameters have the default values

Functions and the scope of its variables

- Functions may contain local variables whose **scope** is only inside a function
 - let, const are block-scope (here, function forms a block)
 - var is function-scope
- Inside script, global execution context (window)
 - variables defined in this context have global-scope (outside a function)
 - Global variables are visible everywhere, not a good practice
- the *scope* of a variable is an important idea in programming

Block-scope with let (1/2)

- Block scope may be inside a function block

```
function getValue(condition) {  
    let color = "blue";  
    if(condition) {  
        // other code  
        return color;  
    }  
    else {  
        // color exists here with value blue  
    }  
    // color exists here with value blue  
}
```

Block-scope with let (2/2)

- Block scope: restricted to inside a statement block - { }

```
function getValue(condition){  
  if(condition){  
    let color = "blue";  
    //other code  
    return color;  
  }  
  else{  
    // color does not exist here  
  }  
  // color does not exist here  
}
```

Global and local scope

```
let myNumber = 5, answer; //global variables
```

```
function square(n) {  
    let b = n * n;  
    return b;  
}
```

scope of n and b is within the curly
brackets of the function only

```
// b, n local variables
```

```
answer = 6 * b ;
```

ERROR: b is only known inside the
function

Global and local scope

```
let myNumber = 5, answer; //global variables
```

```
function square(n){  
    let b = n * n; // b, n local variables  
    return b;  
}
```

ok, so what if I declare b outside the function?

```
let b = 5;  
answer = 6 * b;
```

FINE: but now there are 2 variables called b, one local to the function and one global

Global variables are considered a bad practise because of side-effects.
Side-effects are undesired outcome of global variables changes on the background

Return value

- Function always returns a value. If no return statement is defined, *undefined* is returned

```
function isEven(n) {  
    //function returns value of boolean expr.  
    return (n%2 ==0);  
}  
  
let val = 6;  
if(isEven(val)) {  
    //do something  
}
```

Return value

```
function isEven(n) {  
    if((typeof(n)=="number") && (!isNaN(n))) {  
        return (n%2 ==0);  
    }  
    //else return value is undefined  
}  
  
let val = 5;  
console.log(isEven(val)?"is even":"is odd"); //conditional ternary operator
```

Return value

```
function doSomething (p1)
{
    if (typeof(p1) == "number")
        return 0;          // return a number
    else
        if (typeof(p1) == "string")
            return "zero"; // return a string
}
```

```
doSomething(1);          // returns 0
doSomething("abc");      // returns "zero"
doSomething();           // returns undefined
```

- The same function can return values of different types

Callback functions

- When you pass a function A to another function B, and B executes A, then A is known as a 'callback' function
- Allows programmers to use functions written by others and to provide their own behaviour
- One such situation is event handling

Callback functions

- As a function is 'just' data, it can be passed as a parameter to another function

```
function invokeAdd(a,b){  
    return a() + b();  
}
```

```
function one(){ return 1;}  
function two(){ return 2;}
```

```
answer = invokeAdd(one, two);
```

Self-Invoking functions

Function expressions can be made "self-invoking",
such as a and b in the example:

expression followed by () will be invoked automatically

Arrays as parameters to functions

```
function addArrayItems(arrOfSomething) {  
    let sumOfItems = 0;  
    for (let i = 0; i < arrOfSomething.length; i++) {  
        sumOfItems += arrOfSomething[i];  
    }  
    return sumOfItems;  
}
```

- The parameter is a *reference* to the Array object
- Refer the array parameter using the `length` property to access the individual data items

Changing parameters inside a function

```
function modArrayItems(arrOfSomething) {  
    if(arrOfSomething.length > 0){  
        arrOfSomething[0] = 66;  
    }  
    return;  
}
```

- The parameter is a reference to the array in the calling part of the program
- Changing the reference inside the function changes the *actual* parameter
- (Be very careful)

Changing parameters inside a function

```
let numbers = [3,4,4,5,6,7,9];  
let names = ["pasi","leena","tuomo","kari","tiia"];
```

```
modArrayItems(numbers);  
modArrayItems(names);
```

Console:

```
numbers    Array [ 66, 4, 4, 5, 6, 7, 9 ]  
names Array [ 66, "leena", "tuomo", "kari", "tiia" ]
```

- Changing the parameter inside the function changes the actual parameter